

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date:02-04-2026	Enrolment No:92301733024

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

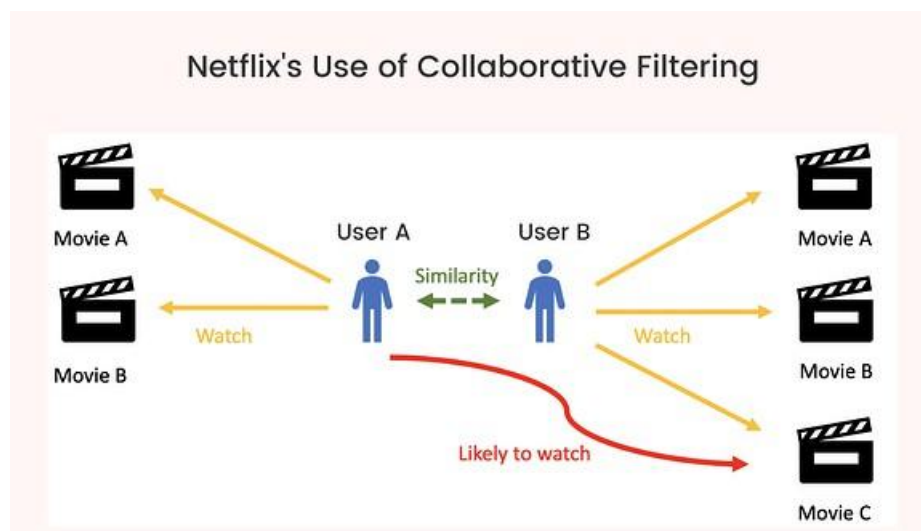
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date:02-04-2026	Enrolment No:92301733024

Pre Lab Exercise:

1. When can you use collaborative-based recommendation system approach?

Collaborative-based recommendation systems are used when there is sufficient user interaction data such as ratings, preferences, or purchase history available. This approach is suitable when we want to recommend items based on the behavior of similar users rather than item features. It is widely used in platforms like Netflix, Amazon, and Spotify where the system identifies users with similar interests and suggests items liked by those users. It is especially useful when item descriptions are not available or are difficult to analyze.

2. Write advantages of collaborative -based filtering approach

- Does not require item features or content
- Can recommend diverse and unexpected items
- Improves as more users interact with the system
- Captures real user preferences effectively
- Suitable for large-scale systems

3. Write disadvantages of collaborative -based filtering approach

- Cold start problem (new users/items)
- Requires large amount of data
- Sparsity problem (many missing ratings)
- Scalability issues with very large datasets
- Can suffer from popularity bias

Program (Code):

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("ratings.csv")
ratings.head()
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date:02-04-2026	Enrolment No:92301733024

userId	movieId	rating	timestamp
0	1	1	4.0 964982703
1	1	3	4.0 964981247
2	1	6	4.0 964982224
3	1	47	5.0 964983815
4	1	50	5.0 964982931

```
# ===== USER-ITEM MATRIX =====
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ===== USER SIMILARITY =====
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ===== RECOMMEND FUNCTION =====
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"
    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]
    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])
    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values
    # Normalize
    weighted_ratings /= similar_users.sum()
    # Get unseen movies
    user Rated = user_item_matrix.loc[user_id]
    unseen_movies = user Rated[user Rated == 0].index
    scores = list(zip(unseen_movies, weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)
    recommended_ids = [movie for movie, _ in scores[:top_n]]
    return recommended_ids
print(recommend_movies(user_id=1))
```

[1200, 1610, 541, 589, 1036]

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date:02-04-2026	Enrolment No:92301733024

Observation:

The system recommends movies based on similarity between users using cosine similarity. Users with similar rating patterns influence recommendations. The model works well when sufficient user data is available, but may struggle with new users or sparse data.

Post Lab Exercise:

1. Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.

The results obtained from the designed model are satisfactory to a certain extent as the system is able to recommend movies based on similar users' preferences, providing a personalized experience. It effectively captures user behavior and suggests relevant items. However, the performance is limited in cases where there is insufficient data, such as for new users or less-rated items. The system may also recommend popular items more frequently, which can reduce the novelty of recommendations.

2. Recommend top-10 "movies" using MovieLens 100K dataset

(Link: <https://grouplens.org/datasets/movielens/100k/>). Attach the code with output.

Also, write your observation for the same.

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("/content/u.data", sep="\t", names=["userId", "movieId", "rating", "timestamp"])
ratings.head()
```

	userId	movieId	rating	timestamp
0	196	242	3	881250949
1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

```
# ===== USER-ITEM MATRIX =====
user_item_matrix = ratings.pivot_table(
    index='userId',
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date:02-04-2026	Enrolment No:92301733024

```

columns='movieId',
values='rating'
).fillna(0)
# ===== USER SIMILARITY =====
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ===== RECOMMEND FUNCTION =====
def recommend_movies(user_id, top_n=10):
    if user_id not in user_item_matrix.index:
        return "User not found!"
    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]
    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])
    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values
    # Normalize
    weighted_ratings /= similar_users.sum()
    # Get unseen movies
    user Rated = user_item_matrix.loc[user_id]
    unseen_movies = user Rated[user Rated == 0].index
    scores = list(zip(unseen_movies, weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)
    recommended_ids = [movie for movie, _ in scores[:top_n]]
    return recommended_ids
print(recommend_movies(user_id=1))

```

[474, 273, 433, 566, 382, 655, 684, 732, 403, 568]

Observation:

The system recommends movies based on preferences of similar users. The recommendations are personalized and effective when enough rating data is available. However, the system may recommend popular items more frequently and struggles with users who have very few ratings.